# Frontend + DSA Interview Guide

JavaScript • React.js • TypeScript • Data Structures & Algorithms

Questions, clear answers, practical examples, pseudocode, and solved problems

# 1. JavaScript

### *Question: What is JavaScript?*

**Answer:** JavaScript is a programming language used to make web applications interactive. It runs in browsers and can also run on servers through Node.js.

**Real example:** A shopping page can update the cart count immediately after Add to Cart is clicked.

**Use case:** UI interactions, browser events, API calls, and backend services.

### *Question: What is the difference between let, const, and var?*

**Answer:** let and const are block scoped. const prevents reassignment of the binding. var is function scoped and has older hoisting behavior.

**Real example:** Use const for a value that will not be reassigned and let for a value that changes.

**Use case:** React components, loops, calculations, and state preparation.

### *Question: What is the difference between == and ===?*

**Answer:** == performs type coercion, while === compares value and type without coercion.

**Real example:** 5 == '5' is true, while 5 === '5' is false.

**Use case:** Use === by default for IDs, flags, status values, and API data.

### *Question: What is a closure?*

**Answer:** A closure lets a function remember variables from the scope where it was created, even after the outer function has finished.

**Real example:** A counter function can keep its count private without exposing the variable directly.

**Use case:** Private state, factories, callbacks, event handlers, and debouncing.

### *Question: What is hoisting?*

**Answer:** JavaScript processes declarations before normal execution. var is initialized as undefined, while let and const stay in the temporal dead zone until their declaration.

**Real example:** A function declaration can be called before its definition, but accessing a let variable before declaration throws an error.

**Use case:** Understanding initialization and execution-order questions.

### *Question: What is this?*

**Answer:** this is determined by how a function is called. In an object method it commonly refers to the object; arrow functions do not create their own this.

**Real example:** A user.getName() method can use this.name.

**Use case:** Objects, classes, event handlers, and libraries.

### *Question: What are map, filter, and reduce?*

**Answer:** map transforms every item, filter keeps matching items, and reduce combines items into one result.

**Real example:** Filter completed orders, map their totals, then reduce them into revenue.

**Use case:** Preparing data before rendering React components or sending API payloads.

### *Question: What is shallow vs deep copy?*

**Answer:** A shallow copy copies the outer object but keeps references to nested objects. A deep copy independently copies nested structures.

**Real example:** {...user} copies the top-level object but user.address can still reference the original nested object.

**Use case:** Avoiding accidental React state mutation.

### *Question: What is a Promise?*

**Answer:** A Promise represents the eventual result of an asynchronous operation and can be pending, fulfilled, or rejected.

**Real example:** fetch() returns a Promise for the server response.

**Use case:** API calls, database operations, and asynchronous workflows.

### *Question: What is async/await?*

**Answer:** async/await is readable syntax for working with Promises. await pauses that async function until the Promise settles.

**Real example:** A dashboard can await /api/orders before rendering returned data.

**Use case:** API integration and readable error handling.

### *Question: What is the event loop?*

**Answer:** Synchronous JavaScript runs on the call stack. Asynchronous work is handled by the runtime and callbacks are scheduled when the stack is available.

**Real example:** A timer callback does not interrupt currently running synchronous code.

**Use case:** Understanding timers, promises, callbacks, and UI responsiveness.

### *Question: What is debouncing?*

**Answer:** Debouncing waits until events stop for a chosen delay before executing a function.

**Real example:** A search box can wait 300 ms after the last keystroke before calling the search API.

**Use case:** Search suggestions, resize handlers, validation, and autosave.

### *Question: What is throttling?*

**Answer:** Throttling limits how often a function can execute during a period.

**Real example:** A scroll handler may run at most once every 100 ms.

**Use case:** Scroll tracking, analytics, and mouse movement.

# 2. React.js

### *Question: What is React?*

**Answer:** React is a library for building user interfaces from reusable components.

**Real example:** A checkout can be split into Cart, AddressForm, Payment, and OrderSummary.

**Use case:** Large interactive web applications.

### *Question: What are props?*

**Answer:** Props are read-only inputs passed from a parent to a child component.

**Real example:** ProductCard receives name, price, and imageUrl.

**Use case:** Reusable components with configurable data.

### *Question: What is state?*

**Answer:** State is component data that changes over time and can trigger a re-render.

**Real example:** A modal can store isOpen in state.

**Use case:** Forms, filters, loading states, and toggles.

### *Question: Props vs state?*

**Answer:** Props come from a parent and should be treated as read-only. State is managed by a state owner and changes through state updates.

**Real example:** A parent passes selectedProduct as a prop while a child owns dropdown-open state.

**Use case:** Deciding where data should live.

### *Question: Why are keys needed in lists?*

**Answer:** Keys give list items stable identity so React can efficiently reconcile additions, removals, and changes.

**Real example:** Use order.id instead of an array index when rendering orders.

**Use case:** Stable list rendering and preserving component state.

### *Question: What is useState?*

**Answer:** useState adds local state to a function component. Updating it schedules a render.

**Real example:** const [quantity,setQuantity] = useState(1).

**Use case:** Counters, forms, filters, selected tabs.

### *Question: Why use functional state updates?*

**Answer:** When new state depends on previous state, the functional form uses the latest state value.

**Real example:** setCount(c => c + 1) is safer for queued updates.

**Use case:** Counters, toggles, and array updates.

### *Question: What is useEffect?*

**Answer:** useEffect runs side effects after a render commits and is useful for synchronization with external systems.

**Real example:** Fetch a profile when userId changes.

**Use case:** API fetching, subscriptions, timers, browser integration.

### *Question: Why does useEffect need cleanup?*

**Answer:** Cleanup stops the previous side effect when dependencies change or when the component unmounts.

**Real example:** Close a WebSocket subscription when the component unmounts.

**Use case:** Preventing duplicate listeners and leaks.

### *Question: useMemo vs useCallback?*

**Answer:** useMemo memoizes a calculated value. useCallback memoizes a function reference.

**Real example:** Memoize an expensive filtered dataset or stabilize a callback passed to a memoized child.

**Use case:** Performance optimization after measurement.

### *Question: What is lifting state up?*

**Answer:** Move shared state to the nearest common parent so siblings use the same source of truth.

**Real example:** Two panels can read the selected shipping address from their parent.

**Use case:** Coordinating sibling components.

### *Question: What is prop drilling?*

**Answer:** Prop drilling means passing data through components that do not need it just to reach a deeper component.

**Real example:** Passing theme through four layers only to reach a button.

**Use case:** Recognizing when Context may simplify data flow.

### *Question: Controlled vs uncontrolled input?*

**Answer:** Controlled inputs keep their value in React state. Uncontrolled inputs keep the current value in the DOM and can be accessed with a ref.

**Real example:** A validated login form is commonly controlled.

**Use case:** Forms, validation, file inputs, and third-party integrations.

### *Question: How would you debug a slow React app?*

**Answer:** Measure first. Determine whether the problem is rendering, expensive calculations, large lists, network latency, or unnecessary updates, then optimize the actual bottleneck.

**Real example:** For 5,000 rows, virtualization may matter more than adding memoization everywhere.

**Use case:** Production performance debugging.

# 3. TypeScript

### *Question: What is TypeScript?*

**Answer:** TypeScript is a statically typed superset of JavaScript that adds compile-time checking and then compiles to JavaScript.

**Real example:** Describe a User object so incorrect property access is caught during development.

**Use case:** Large frontend and backend codebases.

### *Question: What is type inference?*

**Answer:** TypeScript can infer types from values and context, reducing the need for annotations.

**Real example:** const age = 21 is inferred as number.

**Use case:** Readable code with type safety.

### *Question: type vs interface?*

**Answer:** Both can describe object shapes. Interfaces are common for extensible object contracts, while type aliases are especially flexible for unions and intersections.

**Real example:** interface User describes an object; type Status = 'loading' | 'success' | 'error' describes states.

**Use case:** Props, domain models, and API contracts.

### *Question: What is a union type?*

**Answer:** A union says a value can be one of several types or literal values.

**Real example:** type Role = 'admin' | 'user'.

**Use case:** Finite UI and API states.

### *Question: What is a generic?*

**Answer:** Generics let reusable code preserve the relationship between input and output types.

**Real example:** function identity(value:T):T returns the same type it receives.

**Use case:** Utility functions, API wrappers, collections, and components.

### *Question: any vs unknown?*

**Answer:** any disables most checking. unknown is safer because it must be narrowed before use.

**Real example:** Treat external JSON as unknown and validate it before accessing fields.

**Use case:** External input and API boundaries.

### *Question: What is type narrowing?*

**Answer:** Narrowing uses checks such as typeof, equality, or property checks to make a broad type more specific.

**Real example:** After typeof value === 'string', string methods become available.

**Use case:** Union types and external data.

### *Question: What are utility types?*

**Answer:** Partial, Pick, Omit, Record, and Readonly transform existing types.

**Real example:** Partial can represent a partial update payload.

**Use case:** Avoiding duplicated type definitions.

### *Question: How do you type React props?*

**Answer:** Define an interface or type for the component props and use it in the component signature.

**Real example:** ButtonProps can require label:string and allow disabled?:boolean.

**Use case:** Reusable React components.

### *Question: How do you type API responses?*

**Answer:** Define the expected response type at the API boundary. Remember that TypeScript types do not validate runtime JSON, so untrusted data may also need runtime validation.

**Real example:** A /users endpoint can return User[] after validation.

**Use case:** Frontend-backend integration.

# 4. JavaScript vs React vs TypeScript

| Topic | JavaScript | React | TypeScript |
|---|---|---|---|
| What it is | Programming language | UI library | Typed superset of JavaScript |
| Main role | Logic and behavior | UI composition/rendering | Static type checking |
| Example | fetch(), map(), async/await | Component, props, state | interface User, union types |
| Where used | Browser and server | Frontend applications | Frontend/backend projects |

### *Question: If you know JavaScript, why learn TypeScript?*

**Answer:** TypeScript catches many mistakes before runtime and improves maintainability and refactoring in larger projects. It does not replace tests or runtime validation.

**Real example:** If a component expects price:number but receives a string, the mismatch can be highlighted before production.

**Use case:** Team-scale frontend and backend development.

# 5. DSA — Concepts

### *Question: What is Big-O notation?*

**Answer:** Big-O describes how work or memory grows as input size increases. O(1) is constant, O(log n) grows slowly, O(n) is linear, and $O(n^2)$ is quadratic.

**Real example:** Scanning an array is O(n); comparing every pair can be $O(n^2)$.

**Use case:** Comparing scalability of solutions.

### *Question: When should you use an array?*

**Answer:** Use an array for ordered data and fast index access.

**Real example:** A list of products displayed in order is naturally an array.

**Use case:** Sequential data and index-based access.

### *Question: When should you use a hash map?*

**Answer:** Use a hash map when fast average-case lookup by key is needed.

**Real example:** Store userId -> user to avoid scanning all users.

**Use case:** Lookup tables, caching, frequency counts, Two Sum.

### *Question: What is a Set?*

**Answer:** A Set stores unique values and supports fast average-case membership checks.

**Real example:** Insert product IDs into a Set to detect duplicates.

**Use case:** Membership and duplicate detection.

### *Question: What is a stack?*

**Answer:** A stack follows LIFO: last in, first out.

**Real example:** Undo history removes the most recent action first.

**Use case:** Parentheses, undo, DFS, backtracking.

### *Question: What is a queue?*

**Answer:** A queue follows FIFO: first in, first out.

**Real example:** A support queue processes the oldest waiting ticket first.

**Use case:** BFS and task processing.

### *Question: What is recursion?*

**Answer:** Recursion is a function calling itself on a smaller subproblem until a base case.

**Real example:** A tree traversal recursively processes left and right children.

**Use case:** Trees, divide-and-conquer, and backtracking.

# 6. DSA — Basic Problems With Solutions

## *Question: Find the maximum element in an array*

**Answer:** Scan once while storing the largest value seen so far.

**Real example:** [3,7,2,9,5] → 9

**Use case:** Finding the highest transaction or score.

**Short pseudocode:**

```
maximum = arr[0]
for x in arr:
if x > maximum:
maximum = x
return maximum
```

**How the solution works:** Start with the first value and replace it whenever a larger value appears. O(n) time and O(1) extra space.

**Python solution:**

```
def find_max(arr):
maximum = arr[0]
for x in arr:
if x > maximum:
maximum = x
return maximum
```

## *Question: Reverse an array*

**Answer:** Use two pointers at the beginning and end and swap them while moving inward.

**Real example:** [1,2,3,4] → [4,3,2,1]

**Use case:** Reversing ordered records.

**Short pseudocode:**

```
left = 0
right = n - 1
while left < right:
swap arr[left], arr[right]
left += 1
right -= 1
```

**How the solution works:** Every swap puts two elements in their final positions. O(n) time and O(1) extra space.

**Python solution:**

```
def reverse_array(arr):
left, right = 0, len(arr)-1
while left < right:
arr[left], arr[right] = arr[right], arr[left]
left += 1
right -= 1
return arr
```

## *Question: Check whether a string is a palindrome*

**Answer:** Compare characters from both ends toward the center.

**Real example:** 'level' → true; 'hello' → false

**Use case:** Checking symmetric strings.

**Short pseudocode:**

```
left = 0
right = len(s) - 1
while left < right:
if s[left] != s[right]:
return false
left += 1
right -= 1
return true
```

**How the solution works:** If any mirrored pair differs, the string is not a palindrome. O(n) time and O(1) extra space.

**Python solution:**

```
def is_palindrome(s):
left, right = 0, len(s)-1
while left < right:
if s[left] != s[right]:
return False
left += 1
right -= 1
return True
```

## *Question: Count character frequencies*

**Answer:** Use a hash map from character to count.

**Real example:** 'banana' → {b:1,a:3,n:2}

**Use case:** Counting event types, letters, or search terms.

**Short pseudocode:**

```
freq = {}
for ch in s:
freq[ch] = freq.get(ch, 0) + 1
return freq
```

**How the solution works:** Each character updates one map entry. O(n) average time and O(k) space for distinct characters.

**Python solution:**

```
def frequency(s):
freq = {}
for ch in s:
freq[ch] = freq.get(ch, 0) + 1
return freq
```

## *Question: Detect duplicates in an array*

**Answer:** Keep a Set of values already seen; a repeated value means a duplicate exists.

**Real example:** [1,4,2,4] → true

**Use case:** Detecting duplicate IDs or usernames.

**Short pseudocode:**

```
seen = empty set
for x in arr:
if x in seen:
return true
add x to seen
return false
```

**How the solution works:** Average Set membership is O(1), so the overall average time is O(n) and space is O(n).

**Python solution:**

```
def contains_duplicate(arr):
seen = set()
for x in arr:
if x in seen:
return True
seen.add(x)
return False
```

## *Question: Two Sum*

**Answer:** For each number x, check whether target-x has already been seen in a hash map.

**Real example:** [2,7,11,15], target=9 → [0,1]

**Use case:** Finding two values that form a required total.

**Short pseudocode:**

```
seen = {}
for i, x in enumerate(arr):
need = target - x
if need in seen:
return [seen[need], i]
seen[x] = i
return []
```

**How the solution works:** The map turns the partner lookup into average O(1), reducing the usual nested-loop $O(n^2)$ approach to O(n) time with O(n) space.

**Python solution:**

```
def two_sum(arr, target):
seen = {}
for i, x in enumerate(arr):
need = target - x
if need in seen:
return [seen[need], i]
seen[x] = i
return []
```

## *Question: Binary search*

**Answer:** On sorted data, compare the middle value and discard half of the remaining range.

**Real example:** [1,3,5,7,9], target=7 → index 3

**Use case:** Fast lookup in sorted data.

**Short pseudocode:**

```
left = 0
right = n - 1
while left <= right:
mid = (left + right) // 2
if arr[mid] == target:
return mid
if arr[mid] < target:
left = mid + 1
else:
right = mid - 1
return -1
```

**How the solution works:** Each comparison removes roughly half the candidates, giving O(log n) time and O(1) extra space.

**Python solution:**

```
def binary_search(arr, target):
left, right = 0, len(arr)-1
while left <= right:
mid = (left + right)//2
if arr[mid] == target:
return mid
if arr[mid] < target:
left = mid + 1
else:
right = mid - 1
return -1
```

### *Question: Valid parentheses*

**Answer:** Use a stack for opening brackets. Every closing bracket must match the top opening bracket.

**Real example:** '{[()]}' → true; '{[(])}' → false

**Use case:** Validating nested expressions.

**Short pseudocode:**

```
stack = []
for ch in s:
if ch is opening:
push ch
else:
if stack empty or top does not match ch:
return false
pop
return stack is empty
```

**How the solution works:** The most recent unmatched opening bracket must be closed first, which is exactly LIFO. O(n) time and O(n) space.

**Python solution:**

```
def valid_parentheses(s):
pairs = {')':'(', ']':'[', '}':'{'}
stack = []
for ch in s:
if ch in pairs:
if not stack or stack[-1] != pairs[ch]:
return False
stack.pop()
else:
stack.append(ch)
return not stack
```

# 7. DSA — Medium Problems With Solutions

### *Question: Two pointers: remove duplicates from a sorted array*

**Answer:** Use a slow pointer for the next unique position and a fast pointer to scan.

**Real example:** [1,1,2,2,3] → first 3 positions become [1,2,3]

**Use case:** Compacting sorted data in place.

**Short pseudocode:**

```
write = 1
for read from 1 to n-1:
if arr[read] != arr[read-1]:
arr[write] = arr[read]
write += 1
return write
```

**How the solution works:** Because equal values are adjacent, only new values need to be written. O(n) time and O(1) extra space.

**Python solution:**

```
def remove_duplicates(arr):
if not arr: return 0
write = 1
for read in range(1, len(arr)):
if arr[read] != arr[read-1]:
arr[write] = arr[read]
write += 1
return write
```

## *Question: Sliding window: maximum sum of a subarray of size k*

**Answer:** Keep the current window sum; subtract the outgoing value and add the incoming value.

**Real example:** [2,1,5,1,3,2], k=3 → 9

**Use case:** Highest sales or activity over k consecutive periods.

**Short pseudocode:**

```
window = sum(first k values)
best = window
for i from k to n-1:
window += arr[i]
window -= arr[i-k]
best = max(best, window)
return best
```

**How the solution works:** Reuse the previous window instead of summing k elements each time. O(n) time and O(1) extra space.

**Python solution:**

```
def max_sum_k(arr, k):
window = sum(arr[:k])
best = window
for i in range(k, len(arr)):
window += arr[i] - arr[i-k]
best = max(best, window)
return best
```

## *Question: Longest substring without repeating characters*

**Answer:** Use a sliding window and store each character's latest index.

**Real example:** 'abcabcbb' → 3

**Use case:** Finding the longest unique contiguous session or sequence.

**Short pseudocode:**

```
left = 0
best = 0
last = {}
for right, ch in enumerate(s):
if ch in last:
left = max(left, last[ch] + 1)
last[ch] = right
best = max(best, right-left+1)
return best
```

**How the solution works:** Move the left boundary only forward. Each character index is processed a constant number of times, so O(n) time.

**Python solution:**

```
def longest_unique(s):
left = best = 0
last = {}
for right, ch in enumerate(s):
if ch in last:
left = max(left, last[ch]+1)
last[ch] = right
best = max(best, right-left+1)
return best
```

## *Question: Prefix sum: range sum queries*

**Answer:** Precompute cumulative sums so a range can be answered by subtracting two prefix values.

**Real example:** [2,4,1,5], indices 1..3 → 10

**Use case:** Repeated range queries in analytics data.

**Short pseudocode:**

```
prefix[0] = 0
for i from 0 to n-1:
prefix[i+1] = prefix[i] + arr[i]
rangeSum(l,r) = prefix[r+1] - prefix[l]
```

**How the solution works:** Preprocessing is O(n), then each range query is O(1).

**Python solution:**

```
def build_prefix(arr):
prefix = [0]
for x in arr:
prefix.append(prefix[-1] + x)
return prefix

def range_sum(prefix, l, r):
return prefix[r+1] - prefix[l]
```

## *Question: Merge overlapping intervals*

**Answer:** Sort by start time, then merge an interval into the last result when they overlap.

**Real example:** [[1,3],[2,6],[8,10],[9,12]] → [[1,6],[8,12]]

**Use case:** Calendar bookings, maintenance windows, and time ranges.

**Short pseudocode:**

```
sort intervals by start
result = []
for interval in intervals:
if result empty or interval.start > result[-1].end:
append interval
else:
result[-1].end = max(result[-1].end, interval.end)
return result
```

**How the solution works:** Sorting costs O(n log n), followed by an O(n) merge scan, so total time is O(n log n).

**Python solution:**

```
def merge(intervals):
intervals.sort(key=lambda x: x[0])
result = []
for start, end in intervals:
if not result or start > result[-1][1]:
result.append([start, end])
else:
result[-1][1] = max(result[-1][1], end)
return result
```

## *Question: Group anagrams*

**Answer:** Use a normalized word representation as a hash-map key. Sorting characters is the simplest approach.

**Real example:** ['eat','tea','tan','ate','nat','bat'] → grouped anagrams

**Use case:** Grouping keywords or names with identical character composition.

**Short pseudocode:**

```
groups = {}
for word in words:
key = sorted characters of word
groups[key].append(word)
return groups.values()
```

**How the solution works:** Words with the same sorted-character key belong together. With sorting, complexity is commonly O(n*k log k).

**Python solution:**

```
from collections import defaultdict
def group_anagrams(words):
groups = defaultdict(list)
for word in words:
key = ''.join(sorted(word))
groups[key].append(word)
return list(groups.values())
```

## *Question: Product of array except self*

**Answer:** Build prefix products from the left and multiply them by suffix products from the right.

**Real example:** [1,2,3,4] → [24,12,8,6]

**Use case:** Computing a derived metric for each item using all other items.

**Short pseudocode:**

```
output = [1] * n
prefix = 1
for i in 0..n-1:
output[i] = prefix
prefix *= arr[i]
suffix = 1
for i in n-1..0:
output[i] *= suffix
suffix *= arr[i]
return output
```

**How the solution works:** Each output position receives all products to its left and right. O(n) time and O(1) extra space excluding the output array.

**Python solution:**

```
def product_except_self(nums):
out = [1] * len(nums)
prefix = 1
for i in range(len(nums)):
out[i] = prefix
prefix *= nums[i]
suffix = 1
for i in range(len(nums)-1, -1, -1):
out[i] *= suffix
suffix *= nums[i]
return out
```

### *Question: BFS traversal of a graph*

**Answer:** Use a queue to explore nodes level by level and a visited set to avoid repetition.

**Real example:** A → neighbors → next level of neighbors

**Use case:** Shortest path in an unweighted graph and level-order exploration.

**Short pseudocode:**

```
queue = [start]
visited = {start}
while queue not empty:
node = pop_front(queue)
for neighbor in graph[node]:
if neighbor not in visited:
visited.add(neighbor)
queue.push(neighbor)
```

**How the solution works:** Every reachable vertex and edge is processed a bounded number of times, giving O(V+E) with an adjacency list.

**Python solution:**

```
from collections import deque
def bfs(graph, start):
q = deque([start])
visited = {start}
order = []
while q:
node = q.popleft()
order.append(node)
for nei in graph[node]:
if nei not in visited:
visited.add(nei)
q.append(nei)
return order
```

### *Question: DFS traversal of a tree*

**Answer:** Explore one branch deeply before returning, using recursion or an explicit stack.

**Real example:** For a binary tree, visit node, then left subtree, then right subtree.

**Use case:** Tree traversal and branch exploration.

**Short pseudocode:**

```
dfs(node):
if node is null:
return
visit(node)
dfs(node.left)
dfs(node.right)
```

**How the solution works:** Each tree node is visited once, so time is O(n). Recursion uses O(h) call-stack space where h is tree height.

**Python solution:**

```
def dfs(root):
if root is None:
return
print(root.val)
dfs(root.left)
dfs(root.right)
```

# 8. How to Explain Short Pseudocode in an Interview

### *Example: Two Sum*

```
seen = {}
for i, x in enumerate(arr):
need = target - x
if need in seen:
return [seen[need], i]
seen[x] = i
return []
```

**Line 1:** Create a hash map to remember values and their indexes.

**Line 2:** Scan the array once. For each value, keep both its index and value.

**Line 3:** Calculate the complement needed to reach the target.

**Line 4:** Check whether that complement was seen earlier.

**Line 5:** If yes, the stored index and current index form the answer.

**Line 6:** Otherwise remember the current value and its index.

**Interview explanation:** First explain the brute-force $O(n^2)$ idea. Then say that a hash map gives average O(1) lookup, so one pass gives O(n) time at the cost of O(n) extra space.

# 9. DSA Interview Method

### *Question: 1. Clarify the problem*

**Answer:** Ask about constraints, duplicates, sortedness, negative values, empty input, and expected output.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 2. Give the simple approach*

**Answer:** Show the straightforward idea before optimizing it.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 3. Identify the bottleneck*

**Answer:** Explain exactly why the simple approach is slow.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 4. Choose a data structure*

**Answer:** Hash map for lookup, Set for uniqueness, stack for nested/LIFO behavior, queue for BFS/FIFO, pointers for ordered arrays, sliding window for contiguous ranges.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 5. Write short pseudocode*

**Answer:** Keep it language-independent and focus on the decisions.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 6. Code*

**Answer:** Use clear names and handle important edge cases.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 7. Walk through an example*

**Answer:** Trace the variables on a small input.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

### *Question: 8. State complexity*

**Answer:** Give time and extra-space complexity and explain where each comes from.

**Real example:** For Two Sum: clarify the target and expected output, choose a hash map, write pseudocode, code it, test [2,7,11,15] with target 9, and state O(n) time/O(n) space.

**Use case:** Almost every DSA interview.

# 10. Quick DSA Revision

### *Question: Array vs linked list?*

**Answer:** Arrays provide fast index access; linked lists are useful when inserting/removing nodes when you already have the relevant node, but random access is slow.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

### *Question: Stack vs queue?*

**Answer:** Stack is LIFO; queue is FIFO. Think undo history versus a waiting line.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

### *Question: Hash map vs Set?*

**Answer:** A hash map stores key-value pairs; a Set stores unique values.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

### *Question: When can binary search be used?*

**Answer:** When the search space has an ordering that lets you discard a portion after each comparison, commonly a sorted array.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

### *Question: What is a sliding window?*

**Answer:** A moving contiguous range that maintains useful state so the next range can be calculated efficiently.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

### *Question: What is two pointers?*

**Answer:** Two indices move according to a rule, often turning a nested scan into a linear scan.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

### *Question: BFS vs DFS?*

**Answer:** BFS explores level by level using a queue; DFS explores deeply using recursion or a stack.

**Real example:** Choose BFS for level-wise or shortest-path work in an unweighted graph; choose DFS when deep branch exploration is natural.

**Use case:** Algorithm selection in interviews.

# 11. Final Revision Checklist

• JavaScript: scope, closures, this, array methods, promises, async/await, event loop, debouncing, throttling.

• React: components, props, state, hooks, effects, cleanup, keys, lifting state, Context, controlled inputs, performance.

• TypeScript: inference, type/interface, unions, generics, unknown, narrowing, utility types, React props, API types.

• DSA basics: arrays, strings, hash maps, sets, stacks, queues, recursion, Big-O.

• DSA patterns: two pointers, sliding window, prefix sum, binary search, intervals, BFS, DFS.

• Basic solved problems: max element, reverse array, palindrome, frequency, duplicates, Two Sum, binary search, valid parentheses.

• Medium solved problems: fixed-window sum, longest unique substring, range sum, merge intervals, group anagrams, product except self, BFS, DFS.

• For every DSA problem: idea → pseudocode → code → example walkthrough → complexity → edge cases.

# 12. React Props & Patterns Cheat Sheet

Basic → Intermediate → Advanced. Use this section as a last-minute React interview revision sheet.

| Prop / Pattern | What it does | Typical use case |
|---|---|---|
| children | Passes nested JSX/content into a component. | Card, Modal, Layout, Button wrappers. |
| className | Passes CSS class names to an element/component. | Styling reusable components. |
| style | Passes inline style values as an object. | Small dynamic styles; avoid for large style systems. |
| onClick / onChange / onSubmit | Event handler props. | Buttons, inputs, forms. |
| disabled / required / checked | Boolean or form-control props. | Forms and interactive controls. |
| value / defaultValue | Controlled vs initial DOM value. | Controlled forms vs uncontrolled forms. |
| name / id / placeholder | Standard HTML input attributes. | Accessible and usable forms. |
| title / aria-* | Native or accessibility-related attributes. | Tooltips, screen readers, accessible UI. |
| key | Stable identity for list reconciliation. | Rendering arrays; use stable IDs. |
| ref | Access a DOM node or imperative handle. | Focus input, measure element, integrate with DOM APIs. |
| callback prop | Parent passes a function to a child. | Child notifies parent about actions. |
| render prop | Pass a function that returns UI. | Reusable behavior with customizable rendering. |
| spread props {...props} | Forwards multiple props. | Wrapper components and reusable primitives. |
| React.memo | Memoizes a component render based on props. | Avoid unnecessary child renders when props are stable. |
| useMemo | Memoizes a calculated value. | Expensive derived calculations. |
| useCallback | Memoizes a function reference. | Stable callbacks for memoized children or dependencies. |
| forwardRef / ref-as-prop patterns | Expose a DOM ref or imperative API from a reusable component. | Input focus, measurement, UI libraries. |
| Context | Provides shared values without manually passing props through every level. | Theme, locale, auth/session state. |
| Compound components | Several components share implicit state/context. | Tabs, Accordion, Select, Menu APIs. |
| Discriminated union props (TS) | Allows props to change based on a discriminator. | Button variants, async UI states, mutually exclusive props. |

# React Props: practical examples

## Basic component props

```
function Button({ label, disabled, onClick }) {
return (
<button disabled={disabled} onClick={onClick}>
{label}
</button>
);
}
```

Use this when a parent controls what a reusable child displays and what action it performs.

## children

```
function Card({ children }) {
return <div className="card">{children}</div>;
}
```

```
// <Card><h2>Profile</h2><p>...</p></Card>
```

Use children when the component is a reusable container whose inner content should remain flexible.

### Callback prop

```
function SearchBox({ onSearch }) {
return <input onChange={e => onSearch(e.target.value)} />;
}
```

The child owns the UI event; the parent owns the business action. This is one of the most common React interview patterns.

### Spread props

```
function Input({ label, ...inputProps }) {
return (
<label>
{label}
<input {...inputProps} />
</label>
);
}
```

Useful for wrapper components, but do not blindly forward props that should not reach the DOM.

### TypeScript discriminated props

```
type ButtonProps =
| { variant: "link"; href: string; onClick?: never }
| { variant: "button"; onClick: () => void; href?: never };
```

This prevents invalid combinations at compile time, such as a button variant that requires onClick being given only an href.

## 13. TypeScript + React Props Cheat Sheet

| Pattern | Example | Use case |
|---|---|---|
| Required prop | type Props = { name: string } | Component must receive the value. |
| Optional prop | type Props = { disabled?: boolean } | Configurable behavior with a default. |
| Function prop | type Props = { onSave: (id: string) => void } | Child sends an action/event to parent. |
| Children | type Props = { children: React.ReactNode } | Reusable layout/container components. |
| Union prop | variant: 'primary' \| 'danger' | Restrict allowed variants. |
| Generic component | type Props = { data: T[] } | Reusable table/list/select components. |
| Event typing | onChange: React.ChangeEvent | Strongly typed DOM events. |
| State typing | useState(null) | State that may initially be empty. |
| API result | type ApiResponse = { users: User[] } | Typed server responses. |
| Discriminated union | {status:'loading'} \| {status:'success'; data:T} | Model mutually exclusive states. |
| Readonly | Readonly | Prevent accidental prop mutation in types. |

## 14. YouTube Preparation Resources — Mapped to the Guide

These are placed at the end as a quick resource map. For each topic, use the corresponding YouTube search link to find the current version of the video/playlist. The resources below were checked against current listings for freeCodeCamp React/TypeScript interview material and DSA courses. ■cite■turn0search0■turn0search1■turn0search8■

### JavaScript basics

**Watch for:** let const var, closures, hoisting, this, map filter reduce

**YouTube:** https://www.youtube.com/results?search_query=JavaScript+closures+hoisting+this+map+filter+reduce+interview+freeCodeCamp

### JavaScript async

**Watch for:** Promises, async/await, event loop, debounce, throttle

**YouTube:** https://www.youtube.com/results?search_query=JavaScript+promises+async+await+event+loop+debounce+throttle+interview

### React basics

**Watch for:** Components, JSX, props, state, events, lists, keys

**YouTube:** https://www.youtube.com/results?search_query=React+JS+components+props+state+keys+interview+freeCodeCamp

### React interview

**Watch for:** 30 React interview questions covering props, state, hooks, forms, Context, lazy loading, custom hooks and TypeScript integration

**YouTube:** https://www.youtube.com/results?search_query=freeCodeCamp+React+Coding+Interview+Prep+30+Questions+Answers

### React + TypeScript

**Watch for:** Typed components, props, hooks, events, useState, useRef, useReducer

**YouTube:** https://www.youtube.com/results?search_query=freeCodeCamp+React+TypeScript+Course+for+Beginners

### React hooks

**Watch for:** useState, useEffect, useRef, useMemo, useCallback, custom hooks

**YouTube:** https://www.youtube.com/results?search_query=React+hooks+useState+useEffect+useRef+useMemo+useCallback+interview

### Advanced React

**Watch for:** Context, memoization, render props, HOC, lazy loading, code splitting, performance

**YouTube:** https://www.youtube.com/results?search_query=advanced+React+patterns+Context+memo+render+props+HOC+code+splitting

### TypeScript fundamentals

**Watch for:** types, interfaces, unions, generics, unknown, never, utility types

**YouTube:**
https://www.youtube.com/results?search_query=TypeScript+generics+union+interface+unknown+never+utility+types+freeCodeCamp

### DSA fundamentals

**Watch for:** Big O, arrays, strings, stacks, queues, linked lists

**YouTube:** https://www.youtube.com/results?search_query=DSA+fundamentals+Big+O+arrays+strings+stacks+queues+freeCodeCamp

### DSA patterns

**Watch for:** Sliding window, two pointers, binary search, intervals

**YouTube:** https://www.youtube.com/results?search_query=NeetCode+sliding+window+two+pointers+binary+search+intervals

### Trees and graphs

**Watch for:** BFS, DFS, trees, graph traversal

**YouTube:** https://www.youtube.com/results?search_query=NeetCode+trees+graphs+BFS+DFS+interview

### Full DSA course

**Watch for:** Comprehensive interview preparation covering arrays, sliding window, two pointers, sorting, linked lists, stacks, queues, trees, graphs, DP, greedy, intervals and backtracking

**YouTube:** https://www.youtube.com/results?search_query=freeCodeCamp+Data+Structures+Algorithms+Mega+Course+49+Hours

## 15. How to use the videos with this PDF

1. Read the question in this PDF first and try to answer it without looking at the answer.

# 15. Project Ideas + GitHub References

Use these projects after studying the concepts in the guide. The goal is not to copy a repository; use the repository to understand architecture, component structure, TypeScript patterns, API integration, testing, and production practices, then build your own version.

## Project 1 — Task Management Dashboard

**Level:** Beginner → Intermediate

**What to build:** Build a Trello/Jira-style task board with login mock, columns, drag-and-drop, search, filters, priority, due dates and responsive UI.

**Suggested stack:** React + TypeScript + Vite + Tailwind + localStorage

**What you learn:** Practice props, children, forms, controlled inputs, component composition, TypeScript unions, reusable cards and filtering.

**GitHub reference:** https://github.com/tapascript/project-ideas

**Why refer to it:** Use the project-ideas repository for additional project variations and difficulty progression.

## Project 2 — E-commerce Store

**Level:** Intermediate

**What to build:** Build a product catalog with search/filter/sort, product details, cart, wishlist, authentication, checkout mock and order history.

**Suggested stack:** React + TypeScript + Tailwind + React Query/Redux Toolkit + Node/Express

**What you learn:** Practice Context/Redux, typed API responses, reusable components, forms, loading/error states and testing.

**GitHub reference:** https://github.com/alim1496/simple-react-ecommerce

**Why refer to it:** A reference React + TypeScript + Tailwind + Vite e-commerce app with cart, authentication, wishlist and Redux Toolkit.

## Project 3 — Analytics Dashboard

**Level:** Intermediate → Advanced

**What to build:** Build an analytics dashboard with KPI cards, charts, filters, pagination, date ranges, tables, caching and responsive layouts.

**Suggested stack:** React + TypeScript + React Query + Recharts + Chakra/Tailwind

**What you learn:** Practice reusable chart/table components, API state, query caching, filters, pagination, TypeScript props and testing.

**GitHub reference:** https://github.com/danilla-dev/TS-dashboard

**Why refer to it:** Reference dashboard using React, TypeScript, Vite, Chakra UI, React Query, charts, tables, filtering and testing.

## Project 4 — Full-Stack SaaS Starter

**Level:** Advanced

**What to build:** Build a small SaaS product such as an issue tracker, team workspace or developer portal with authentication, roles, CRUD, API validation and a database.

**Suggested stack:** React + TypeScript + Express + PostgreSQL/Prisma + authentication

**What you learn:** Practice frontend-backend integration, API contracts, shared types, authentication, authorization, database design and deployment.

**GitHub reference:** https://github.com/markryandelarmente/full-stack-starter-kit

**Why refer to it:** Reference monorepo using React, Express, TypeScript, Tailwind, PostgreSQL, Prisma, authentication and shared packages.

## Project 5 — Production E-commerce Platform

**Level:** Advanced

**What to build:** Build a complete store with storefront, admin panel, product CRUD, cart, checkout flow, reviews, authentication, search/filtering and a design system.

**Suggested stack:** Next.js/React + TypeScript + NestJS + PostgreSQL + React Query + Tailwind

**What you learn:** Study production architecture, reusable design-system components, typed SDKs, admin workflows, authentication and API boundaries.

**GitHub reference:** https://github.com/viniarruda/react-ecommerce

**Why refer to it:** A large reference e-commerce monorepo with storefront, admin panel, REST API, design system and typed client.

### Project 6 — Full-Stack TypeScript Application

**Level:** Advanced architecture study

**What to build:** Build a full-stack application where the frontend and backend share strong TypeScript contracts. Add authentication, validation, tests, CI/CD and observability.

**Suggested stack:** React + TypeScript + Node.js/Express + database + Docker

**What you learn:** Practice architecture, shared types, API design, SSR/client rendering concepts, testing and deployment.

**GitHub reference:** https://github.com/FlorianRappl/typescript-fullstack-sample

**Why refer to it:** A compact reference demonstrating a TypeScript full-stack web app using Node.js and React, including client/server rendering and global state.

## 16. Project Ideas Specifically for Your Interview Preparation

If your goal is a software-engineering interview, build projects that force you to explain frontend architecture, API integration, TypeScript, testing and performance—not just the visual UI.

### A. Merchant / Business Management Dashboard

**Idea:** Create a dashboard where a business owner can manage stores, products, employees, orders and notifications.

**Stack:** React + TypeScript + Node.js + Express + MongoDB/PostgreSQL

**Interview value:** Reusable tables, filters, role-based access, API integration, pagination, optimistic updates, error states, Playwright tests.

### B. AI Resource Generator for Teachers

**Idea:** A teacher enters a topic and grade level; the app generates a lesson outline, quiz questions, diagrams/resources and downloadable material.

**Stack:** React + TypeScript + Node.js + Gemini/OpenAI API + database

**Interview value:** Streaming responses, API abstraction, prompt management, validation, rate limits, file generation and clean component architecture.

### C. Developer Job Tracker

**Idea:** Track applications, companies, interview stages, follow-ups, deadlines and notes with analytics.

**Stack:** React + TypeScript + Node.js + PostgreSQL

**Interview value:** Forms, discriminated status types, filtering, charts, reminders, authentication, CRUD APIs and database design.

### D. Observability Dashboard

**Idea:** Create a mini platform that ingests application logs and displays error rates, latency, request volume and recent incidents.

**Stack:** React + TypeScript + Node.js + Elasticsearch/Kibana-style data model

**Interview value:** Data visualization, pagination, search, aggregation APIs, debouncing, caching and performance.

### E. Telemedicine / Appointment Platform

**Idea:** Build doctor discovery, appointment booking, patient dashboard, consultation history and notifications. Keep sensitive/regulated functionality as a mock/demo.

**Stack:** React + TypeScript + Node.js + MongoDB/PostgreSQL

**Interview value:** Complex forms, calendars, role-based UI, API design, reusable components, validation, testing and system design.

# 17. What to Add to Every Project Before Putting It on Your Resume

• **Authentication:** login/logout and protected routes where appropriate.

• **API layer:** clear REST endpoints, validation, loading/error states and consistent responses.

• **TypeScript:** typed props, API models, unions/discriminated unions and reusable generic components where useful.

• **Testing:** unit/component tests plus at least a few Playwright end-to-end flows.

• **Performance:** pagination or virtualization for large lists, debouncing for search, caching for repeated API data.

• **Security:** validate input, protect secrets, avoid trusting client-side authorization.

• **Observability:** structured logs, useful error messages and basic metrics.

• **Deployment:** production build, environment variables, CI/CD and a live demo if possible.

• **Documentation:** README with architecture, setup, API endpoints, screenshots, trade-offs and future improvements.

• **Interview preparation:** be able to explain one difficult technical decision, one bug you solved, one performance improvement and one trade-off.

# 18. GitHub Study Method

For each reference repository, do not clone it and immediately copy features. First inspect the README and folder structure, identify the main architectural decisions, run it locally, trace one user flow from UI → API → database, and then rebuild a smaller version yourself.

**Suggested progression:** Task dashboard → E-commerce → Analytics dashboard → Full-stack SaaS → Production-style e-commerce/monorepo.

The linked repositories are reference implementations; their architecture, dependencies and maintenance status can change over time. Use them for study and verify current setup instructions in each repository's README.